Aufgabe 5: Großstadtbauern

Team-ID: 00562

Team-Name: Vikkrck07

Bearbeiter/-innen dieser Aufgabe:
Kristof Leidig

16. November 2025

Inhaltsverzeichnis

1 Lösungsidee **1**
1.1 Google-Or-Tools 1

2 Umsetzung **2**
2.1 Entscheidungsvariablen 2
2.2 Constraints 2
2.3 Optimierungsziel 2

3 Werkzeuge **2**

4 Beispiele **3**

5 Quellcode **6**
5.1 Hauptprogramm 6

1 Lösungsidee

Die Lösung basiert auf einem Constraint Programming-Ansatz mit Google OR-Tools. Die Kernidee ist die Modellierung des Problems als constraint satisfaction problem mit folgenden Hauptkomponenten:

- **Anbauplanung**: Jede der drei verfügbaren Anbauflächen kann in jedem Monat höchstens eine Pflanze anbauen
- **Wachstumszyklen**: Pflanzen benötigen drei aufeinanderfolgende Monate zum Wachsen (zyklisch über das Jahr)
- **Gerichte-Zubereitung**: Ein Gericht kann nur zubereitet werden, wenn alle benötigten Zutaten im selben Monat verfügbar sind
- **Ziel**: Maximierung der "Gesamtpunktzahl"durch Zubereitung von Gerichten

1.1 Google-Or-Tools

Warum habe ich mich für Google-Or-Tools entschieden? Zum Anfang sind folgende Möglichkeiten gegeben.

- **Brute-Force**: Schlichtes Ausprobieren aller Möglichkeiten über bspw. Rekursives Backtracking. Dies ist schlecht weil es eine Ewigkeit dauert bis eine gute Lösung oder die Beste Lösung gefunden wird und ggf. wird der Stack gänzlich gefüllt.

- **Heuristiken**: Man könnte ein Prinzip entwickeln diese Daten zu klassifizieren und irgendwie eine möglichst gute Lösung in annehmbarer Zeit finden. (bspw. meist verwendete Zutaten öä.) Problem hierbei ist die unbekannte Güte des Ergebnisses.
- **Google-Or-Tools**: Erlaubt es einem das Problem in Variablen auszudrücken und einem Lösungsschema zu übergeben. Natürlich macht man es sich damit gewissermaßen einfach, sollte man dieses Tool bereits kennen. Zum Anfang kannte ich es jedoch nicht, wesshalb viel Lesen der Docs nötig war um das Verständniss zu entwickeln und die Variablen richtig zu bestimmen.

2 Umsetzung

Die Umsetzung erfolgte in Python mit dem OR-Tools Constraint Programming Solver. Die wichtigsten Implementierungsschritte waren:

2.1 Entscheidungsvariablen

- Boolean Variable, ob eine Pflanze auf Fläche im Monat x startet
- Boolean Variable, ob eine Pflanze auf Fläche im Monat x wächst
- Boolean Variable, ob Gericht y zubereitet wird
- Boolean Variable, ob Gericht im Monat x zubereitet werden kann

2.2 Constraints

- **Wachstumsdauer**: Wenn eine Pflanze im Monat x startet, muss sie in den Monaten x, x+1, x+2 wachsen
- **Keine Überlappung**: Keine zwei Pflanzen dürfen auf derselben Fläche überlappende Wachstumszyklen haben
- **Keine Überlappung 2**: Jede Fläche kann pro Monat höchstens eine Pflanze anbauen
- **"Kochbarkeit"**: Ein Gericht kann nur zubereitet werden, wenn alle Zutaten im selben Monat verfügbar sind

2.3 Optimierungsziel

Maximierung der Summe der Punkte aller zubereiteten Gerichte.

3 Werkzeuge

- **Python 3**: Hauptprogrammiersprache
- **Google OR-Tools**: Constraint Programming Solver für die optimale Planung
- **KI (Deepseek, Claude)**: Unterstützung bei der Erstellung dieser Dokumentation

Vorgehensweise mit den Werkzeugen:

1. Zuerst wurde das Problem analysiert und in ein constraint satisfaction problem umgewandelt. Hierbei ist auf die Dokumentation von Google-Or-Tools zu verweisen, die sehr hilfreich waren :).
2. Die Google-Or Bibeliothek wurde über pip installiert (unbedingt Tuen vor der Programmausführung! -> Anleitung in den Docs von Google-Or)
3. Die KI hat geholfen Rechtschreibfehler in dieser Dokumentation zu vermeiden
4. Die Ausgabe wurde von Claude formatiert, damit das resultat besser lesbar ist. Dies halte ich für vertretbar, da eine bessere Repräsentierung nicht der Kern der Aufgabe ist.

4 Beispiele

1. Beispiel 1: "bauern1.txt"

```
Crops: ['Kartoffel', 'Zwiebel', 'Möhre', 'Reis']
Dishes: 4
Optimal Solution/Time limit not reached

Total dishes: 4.0
Time(in s): 0.034

============================================================
COMPLETED DISHES
============================================================
Dish 1: Kartoffel, Zwiebel, Möhre
  Reward: 1 points | Completed in: Nov
Dish 2: Kartoffel, Zwiebel, Reis
  Reward: 1 points | Completed in: Sep
Dish 3: Kartoffel, Möhre, Reis
  Reward: 1 points | Completed in: Oct
Dish 4: Zwiebel, Möhre, Reis
  Reward: 1 points | Completed in: Dec

Total Reward: 4.0

============================================================
PLANTING SCHEDULE
============================================================

Slots     Jan         Feb         Mar         Apr         May         Jun
------------------------------------------------------------------------
Slot 1    -           -           -           Möhre       Möhre       Möhre
Slot 2    Zwiebel     Reis        Reis        Reis        Reis        Reis
Slot 3    Reis        Reis        Möhre       Möhre       Möhre       -

          Jul         Aug         Sep         Oct         Nov         Dec
------------------------------------------------------------------------
          Zwiebel     Zwiebel     Zwiebel     Möhre       Möhre       Möhre
          Reis        Reis        Reis        Reis        Zwiebel     Zwiebel
          -           -           Kartoffel   Kartoffel   Kartoffel   Reis
```

   Dieses Beispiel wird optimal gelöst und es muss nichteinmal der gesamte Plan gefüllt werden. Die Ausführungszeit ist sehr gering aber das ist zu erwarten. Große Probleme hat dieses Beispiel nicht bereitet, es war mehr oder minder der Funktionalitätstest.

2. Beispiel 2: "bauern2.txt"

```
Crops: ['Kartoffel', 'Zwiebel', 'Möhre', 'Reis', 'Paprika', 'Tomate', 'Petersilie'
, 'Champignons']
Dishes: 6
Optimal Solution/Time limit not reached

Total dishes: 6.0
Time(in s): 0.088

============================================================
COMPLETED DISHES
============================================================
Dish 1: Kartoffel, Zwiebel, Möhre
```

```
  Reward: 1 points | Completed in: Apr
Dish 2: Tomate, Zwiebel, Petersilie
  Reward: 1 points | Completed in: Jul
Dish 3: Paprika, Zwiebel, Tomate
  Reward: 1 points | Completed in: Aug
Dish 4: Reis, Champignons, Zwiebel
  Reward: 1 points | Completed in: Nov
Dish 5: Kartoffel, Paprika, Zwiebel
  Reward: 1 points | Completed in: Feb
Dish 6: Reis, Paprika, Tomate
  Reward: 1 points | Completed in: Sep

Total Reward: 6.0

============================================================
PLANTING SCHEDULE
============================================================

Slots     Jan         Feb         Mar         Apr         May         Jun
------------------------------------------------------------------------
Slot 1    Kartoffel   Kartoffel   Kartoffel   Möhre       Möhre       Möhre
Slot 2    Champignons Zwiebel     Zwiebel     Zwiebel     Petersilie  Petersilie
Slot 3    Paprika     Paprika     Kartoffel   Kartoffel   Kartoffel   Zwiebel

          Jul         Aug         Sep         Oct         Nov         Dec
------------------------------------------------------------------------
          Tomate      Tomate      Tomate      Zwiebel     Zwiebel     Zwiebel
          Petersilie  Paprika     Paprika     Paprika     Champignons Champignons
          Zwiebel     Zwiebel     Reis        Reis        Reis        Paprika
```

Auch dieses Beispiel benötigt minimale Zeit und ist damit kaum eine Herausforderung. Anzumerken ist aber hier ist der gesamte Zeitplan gefüllt.

3. Beispiel 3: "bauern3.txt"

```
Crops: ['Apfel', 'Avocado', 'Birne', 'Blumenkohl', 'Bohnen', 'Brokkoli',
        'Champignons', 'Erbse', 'Erdbeeren', 'Feige', 'Fenchel', 'Gurke',
        'Himbeeren', 'Kartoffel', 'Kichererbse', 'Knoblauch', 'Kohlrabi',
        'Lauch', 'Linse', 'Mandarine', 'Möhre', 'Olive', 'Paprika',
        'Pastinake', 'Pilze', 'Porree', 'Quitte', 'Rhabarber', 'Rosenkohl',
        'Rote Bete', 'Schwarzwurzel', 'Sellerie', 'Spinat', 'Stachelbeere',
        'Tomate', 'Weintrauben', 'Zucchini', 'Zwiebel']
Dishes: 19
Optimal Solution/Time limit not reached

Total dishes: 8.0
Time(in s): 34.017

============================================================
COMPLETED DISHES
============================================================
Dish 4: Kartoffel, Pastinake, Rosenkohl
  Reward: 1 points | Completed in: Nov
Dish 9: Möhre, Paprika, Tomate
  Reward: 1 points | Completed in: Jul
Dish 12: Apfel, Kartoffel, Tomate
  Reward: 1 points | Completed in: Sep
Dish 13: Kartoffel, Zwiebel, Möhre
  Reward: 1 points | Completed in: May
```

```
Dish 14: Kartoffel, Zwiebel, Paprika
  Reward: 1 points | Completed in: Apr
Dish 15: Kartoffel, Zucchini, Paprika
  Reward: 1 points | Completed in: Mar
Dish 16: Linse, Zucchini, Paprika
  Reward: 1 points | Completed in: Feb
Dish 17: Linse, Zucchini, Pastinake
  Reward: 1 points | Completed in: Jan

Total Reward: 8.0

============================================================
PLANTING SCHEDULE
============================================================

Slots     Jan         Feb         Mar         Apr         May         Jun
------------------------------------------------------------------------
Slot 1    Zucchini    Zucchini    Zucchini    Zwiebel     Zwiebel     Zwiebel
Slot 2    Pastinake   Paprika     Paprika     Paprika     Möhre       Möhre
Slot 3    Linse       Linse       Kartoffel   Kartoffel   Kartoffel   Paprika

          Jul         Aug         Sep         Oct         Nov         Dec
------------------------------------------------------------------------
          Tomate      Tomate      Tomate      Rosenkohl   Rosenkohl   Rosenkohl
          Möhre       Apfel       Apfel       Apfel       Pastinake   Pastinake
          Paprika     Paprika     Kartoffel   Kartoffel   Kartoffel   Linse
```

Das erste Beispiel was eine längere Zeit benötigt. Hier wird zwar noch eine Optimale Lösung gefunden aber auf meinem Endgerät (ryzen 7700) dauert es 35 Sekunden.

4. Beispiel 4: "bauern4.txt"

```
Crops: ['Rote_Bete', 'Champignons', 'Avocado', 'Knoblauch', 'Kichererbse',
        'Kartoffel', 'Brokkoli', 'Porree', 'Bohnen', 'Reis', 'Nektarine',
        'Banane', 'Weizen', 'Zwiebel', 'Himbeeren', 'Pflaume', 'Radieschen',
        'Linse', 'Olive', 'Pastinake', 'Schwarzwurzel', 'Zucchini', 'Pilze',
        'Kohlrabi', 'Spinat', 'Apfel', 'Quitte', 'Melone', 'Feige', 'Rhabarber',
        'Rosenkohl', 'Birne', 'Gurke', 'Mais', 'Pfirsich', 'Kiwi', 'Paprika',
        'Erbse', 'Möhre', 'Tomate', 'Blumenkohl', 'Granatapfel', 'Lauch',
        'Maracuja', 'Mirabelle', 'Fenchel', 'Linsen', 'Erdbeeren', 'Mandarine',
        'Weintraube']
Dishes: 36
Some Solution/Time limit reached

Total dishes: 9.0
Time(in s): 60.065

============================================================
COMPLETED DISHES
============================================================
Dish 23: Apfel, Kartoffel, Paprika
  Reward: 1 points | Completed in: Mar
Dish 24: Apfel, Kohlrabi, Paprika
  Reward: 1 points | Completed in: Feb
Dish 25: Banane, Kohlrabi, Paprika
  Reward: 1 points | Completed in: Jan
Dish 26: Banane, Kohlrabi, Zwiebel
  Reward: 1 points | Completed in: Dec
Dish 28: Apfel, Kartoffel, Tomate
```

```
  Reward: 1 points | Completed in: Apr
Dish 30: Kartoffel, Zwiebel, Paprika
  Reward: 1 points | Completed in: Oct
Dish 31: Kartoffel, Zucchini, Paprika
  Reward: 1 points | Completed in: Sep
Dish 32: Linsen, Zucchini, Paprika
  Reward: 1 points | Completed in: Aug
Dish 33: Linsen, Zucchini, Pastinake
  Reward: 1 points | Completed in: Jul

Total Reward: 9.0

============================================================
PLANTING SCHEDULE
============================================================

Slots     Jan         Feb         Mar         Apr         May         Jun
------------------------------------------------------------------------
Slot 1    Paprika     Paprika     Paprika     Tomate      Tomate      Tomate
Slot 2    Banane      Apfel       Apfel       Apfel       Pastinake   Pastinake
Slot 3    Kohlrabi    Kohlrabi    Kartoffel   Kartoffel   Kartoffel   Linsen

          Jul         Aug         Sep         Oct         Nov         Dec
------------------------------------------------------------------------
          Zucchini    Zucchini    Zucchini    Zwiebel     Zwiebel     Zwiebel
          Pastinake   Paprika     Paprika     Paprika     Banane      Banane
          Linsen      Linsen      Kartoffel   Kartoffel   Kartoffel   Kohlrabi
```

Das ist das erste Beispiel, bei dem das Zeitlimit ausgereizt wird (wenn es auf 60.0 steht). Hier wird angemerkt, dass dies passiert ist und die bis dahin beste Lösung wird zurückgegeben. Das Limit kann aber erhöht werden und eine Optimale Lösung ist noch in Aussicht.

5. Beispiel 5/6:

   Hierfür werde ich erstmal keine Ausgaben bereitstellen, da die Beispiele optional waren, aber erfahrungsgemäß werden hier je nach Zeitlimit Lösungen von 8-10 Gerichten gefunden. Auch sind die Beispiele hier nicht zu finden um zu Verhindern, dass dieses Dokument aus ausschließlich Konsolenausgaben besteht, aber der Code liefert Ergebnisse!

5 Quellcode

5.1 Hauptprogramm

```
import sys
from ortools.sat.python import cp_model
from reader import read_file

# constants
MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
NUM_MONTHS = 12
PLANT = 3
CYCLES = 3

#read the file
filename = sys.argv[0]
crops, dishes = read_file(filename)

#get next 3 month :: [3 months]
def occ_months(start):
    return [(start + i) % NUM_MONTHS for i in range(CYCLES)]

#check overlap :: bool
def overlap(start1, start2):
```

```
    return len(set(occ_months(start1)) & set(occ_months(start2))) > 0

print(f"Crops: {crops}")
print(f"Dishes: {len(dishes)}")

# create solver
model = cp_model.CpModel()

# Decision variable start of cycle
starty = {}
for plant in range(PLANT):
    starty[plant] = {}
    for month in range(NUM_MONTHS):
        starty[plant][month] = {}
        for crop in crops:
            var_name = f"slot:{plant}+month:{month}+ingredient{crop}"
            starty[plant][month][crop] = model.NewBoolVar(var_name)

# decision variable whole cycle
growing = {}
for plant in range(PLANT):
    growing[plant] = {}
    for month in range(NUM_MONTHS):
        growing[plant][month] = {}
        for crop in crops:
            var_name = f"slot:{plant}+month:{month}+ingredient{crop}"
            growing[plant][month][crop] = model.NewBoolVar(var_name)

# Decision variable cookable dish
dish = []
for i in range(len(dishes)):
    var_name = f"dish:{i}"
    dish.append(model.NewBoolVar(var_name))

# starty is first of growing
for plant in range(PLANT):
    for s in range(NUM_MONTHS):
        for crop in crops:
            occupied_months = occ_months(s)
            for month in occupied_months:
                model.AddImplication(starty[plant][s][crop],growing[plant][month][crop])

# impose length of 3
for plant in range(PLANT):
    for month in range(NUM_MONTHS):
        for crop in crops:
            poss_start = [(month - i) % NUM_MONTHS for i in range(CYCLES)]
            model.Add(sum(starty[plant][start][crop] for start in poss_start) >=
            1).OnlyEnforceIf(growing[plant][month][crop])
            model.Add(sum(starty[plant][start][crop] for start in poss_start) ==
            0).OnlyEnforceIf(growing[plant][month][crop].Not())

# no simultaneous start
for month in range(NUM_MONTHS):
    curr_plants = []
    for plant in range(PLANT):
        for crop in crops:
            curr_plants.append(starty[plant][month][crop])
    model.Add(sum(curr_plants) <= 1)

# no same plants
for plant in range(PLANT):
    for month in range(NUM_MONTHS):
        model.Add(sum(growing[plant][month][crop] for crop in crops) <= 1)

# avoid multiple of the same crops being grown
for plant in range(PLANT):
    for s_month1 in range(NUM_MONTHS):
        for s_month2 in range(s_month1 + 1, NUM_MONTHS):
            if overlap(s_month1, s_month2):
                for crop1 in crops:
                    for crop2 in crops:
                        model.AddBoolOr([starty[plant][s_month1][crop1].Not(), starty[plant][s_month2]
```

```
                        [crop2].Not()])

#count imposed constraints
constraint_c = 0
constraint_c += PLANT * NUM_MONTHS * len(crops) * CYCLES
constraint_c += PLANT * NUM_MONTHS * len(crops) * 2
constraint_c += NUM_MONTHS
constraint_c += PLANT * NUM_MONTHS

# Decision variable crop is available
available = {}
for month in range(NUM_MONTHS):
    available[month] = {}
    for crop in crops:
        var_name = f"available:{month}+{crop}"
        available[month][crop] = model.NewBoolVar(var_name)

# add available condition
for month in range(NUM_MONTHS):
    for crop in crops:
        curr_grow = [growing[slot][month][crop] for slot in range(PLANT)]
        model.Add(sum(curr_grow) >= 1).OnlyEnforceIf(available[month][crop])
        model.Add(sum(curr_grow) == 0).OnlyEnforceIf(available[month][crop].Not())

# Decision variable dish can be made in given month
dish_completable_in = {}
for dish_ind in range(len(dishes)):
    dish_completable_in[dish_ind] = {}
    for month in range(NUM_MONTHS):
        var_name = f"dish:{dish_ind}+month:{month}"
        dish_completable_in[dish_ind][month] = model.NewBoolVar(var_name)

# check all ingredients available
for dish_ind, (reward, ing) in enumerate(dishes):
    for month in range(NUM_MONTHS):
        ing_month = [available[month][ing] for ing in ing]
        model.Add(sum(ing_month) == len(ing)).OnlyEnforceIf(dish_completable_in[dish_ind][month])
        model.Add(sum(ing_month) < len(ing)).OnlyEnforceIf(dish_completable_in[dish_ind][month].Not())

# set max of 1
for dish_ind in range(len(dishes)):
    model.Add(sum(dish_completable_in[dish_ind][month] for month in range(NUM_MONTHS)) <= 1)

# connect to completable in
for dish_ind in range(len(dishes)):
    model.Add(sum(dish_completable_in[dish_ind][month] for month in range(NUM_MONTHS)) >=
    1).OnlyEnforceIf(dish[dish_ind])
    model.Add(sum(dish_completable_in[dish_ind][month] for month in range(NUM_MONTHS)) ==
    0).OnlyEnforceIf(dish[dish_ind].Not())

# maximise reward/dishes
to_max = []
for dish_ind, (reward, ingredients) in enumerate(dishes):
    to_max.append(reward * dish[dish_ind])

model.Maximize(sum(to_max))

# solve
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 60.0  # time limit (can be variable)
solver.parameters.log_search_progress = False  # Show progress off for submission
solver.parameters.num_search_workers = 16  # available threads (maximise if possible)
status = solver.Solve(model)

# optimal?
if status == cp_model.OPTIMAL: print("Optimal Solution/Time limit not reached")
elif status == cp_model.FEASIBLE: print("Some Solution/Time limit reached")
elif status == cp_model.INFEASIBLE: print("No dishes can be cooked") # should not occur
elif status == cp_model.MODEL_INVALID: print("Something went wrong")
else: print(f"{status}") #something else happened (debugging)

if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f"\nTotal dishes: {solver.ObjectiveValue()}")
```

```
    print(f"Time(in s): {solver.WallTime():.3f}")

    # Here starts the output
    # It is created with AI as explained in the Docs
    # The comments are added by hand to explain the steps taken by the AI
    print("\n" + "=" * 60)
    print("COMPLETED DISHES")
    print("=" * 60)

    # list all dishes that have been completed
    completed_dishes_list = []
    for dish_ind, (reward, ingredients) in enumerate(dishes):
        if solver.Value(dish[dish_ind]):
            completion_month = None
            for month in range(NUM_MONTHS):
                if solver.Value(dish_completable_in[dish_ind][month]):
                    completion_month = month
                    break

            completed_dishes_list.append({
                'dish_num': dish_ind + 1,
                'reward': reward,
                'ingredients': ingredients,
                'month': completion_month
            })

    if completed_dishes_list:
        for dish_info in completed_dishes_list:
            month_name = MONTHS[dish_info['month']] if dish_info['month'] is not None else "Unknown"
            print(f"Dish {dish_info['dish_num']}: {', '.join(dish_info['ingredients'])}")
            print(f"  Reward: {dish_info['reward']} points | Completed in: {month_name}")
    else:
        print("No dishes completed")

    print(f"\nTotal Reward: {solver.ObjectiveValue()}")

    # Create schedule visualization aka. timetable with crops
    print("\n" + "=" * 60)
    print("PLANTING SCHEDULE")
    print("=" * 60)

    # make a data structure containing all info
    schedule = {slot: {month: None for month in range(NUM_MONTHS)} for slot in range(PLANT)}

    for plant in range(PLANT):
        for month in range(NUM_MONTHS):
            for crop in crops:
                if solver.Value(growing[plant][month][crop]):
                    schedule[plant][month] = crop
                    break

    # header
    print("\n{:<10}".format("Slots"), end="")
    for month_name in MONTHS:
        print(f"{month_name:<12}", end="")
    print()
    print("-" * (10 + 12 * NUM_MONTHS))

    # insert the plants
    for plant in range(PLANT):
        print(f"Slot {plant + 1:<5}", end="")
        for month in range(NUM_MONTHS):
            crop_name = schedule[plant][month] if schedule[plant][month] else "-"
            print(f"{crop_name:<12}", end="")
        print()


else:
    print("\nNo solution.") #aka error
```

PS: Die Ausführung erfolgt über die Konsole und muss python bauer.py [insert filename from input] sein!